



Что мы будем делать

Сейчас я продолжу работать с учебным проектом. Его корень далее обозначается переменной `$projectRoot`:

```
<каталог проекта>
```

Но сами задания я уже не буду запускать обычными сообщениями в чате. Я создам отдельную очередь Hermes Kanban.

В этой очереди будет три независимых задания. Я специально назначу им разные приоритеты и ограничу систему одним одновременно работающим агентом. Благодаря этому мы сможем увидеть настоящую очередь: одна задача выполняется, остальные остаются в состоянии `ready`.

Затем я изменю приоритет одной карточки, проверю, что она поднялась в очереди, запущу dispatcher и буду наблюдать переходы между состояниями непосредственно через веб-интерфейс Hermes.

Проверяем профиль, который будет выполнять задачи

Сначала открываю обычный PowerShell Windows.

Открываю PowerShell в корне выбранного проекта и сохраняю вычисленный путь:

```
$projectRoot = (Get-Location).Path
```

Проверяю существующие Hermes-профили:

```
hermes profile list
```

Активный профиль отмечен символом `*`.

Для этой работы я создам отдельного рабочего агента, чтобы очередь не зависела от текущего интерактивного чата.

Выполняю:

```
hermes profile create process-worker --clone --description "Выполняет короткие read-only задачи анализа программного проекта и возвращает проверяемый результат."
```

Параметр `--clone` копирует конфигурацию, credentials, skills и основные настройки из активного профиля, поэтому нам не требуется заново настраивать модель и провайдера. Hermes поддерживает отдельные именованные профили именно для таких независимых работников Kanban.

Проверяю профиль:

```
hermes profile show process-worker
```

В результате я хочу увидеть настроенную модель и существующую конфигурацию.

Если профиль `process-worker` уже существует после предыдущего запуска практики, второй раз создавать его не нужно.

Создаём отдельную очередь для занятия

Чтобы наши учебные задания не смешивались с другими Канбан-задачами, создаю отдельную доску.

Выполняю:

```
hermes kanban boards create process-lab --name "Process Lab" --description "Учебная очередь для практики управления агентными процессами" --switch
```

Если Hermes сообщает, что такая доска уже существует, просто переключаюсь на неё:

```
hermes kanban boards switch process-lab
```

Проверяю:

```
hermes kanban boards show
```

Активной должна быть:

```
process-lab
```

Hermes позволяет иметь отдельные Kanban boards для разных проектов или областей работы. У каждой такой доски своя очередь и собственная база задач.

Ограничиваем параллельность

Теперь я специально запрещаю Hermes выполнять несколько карточек одновременно.

Выполняю:

```
hermes config set kanban.max_in_progress 1
```

Проверяю значение:

```
hermes config get kanban.max_in_progress
```

Должно быть:

```
1
```

Это означает, что dispatcher разрешит одновременно только одну задачу со статусом `running`.
Остальные будут ждать освобождения ресурса в `ready`.

Hermes поддерживает `kanban.max_in_progress` именно как ограничение общего количества одновременно работающих Kanban workers. Если лимит уже достигнут, новые задачи остаются в очереди.

Пока не запускаем dispatcher

Это важно.

Сейчас я сначала заполню очередь и только потом разрешу агентам её разбирать.

Если gateway уже запущен в другом PowerShell и активно выполняет Kanban-задачи, на время демонстрации можно остановить его:

```
hermes gateway stop
```

Если он не был запущен, ничего делать не нужно.

Проверить состояние можно командой:

```
hermes gateway status
```

Нам сейчас нужно спокойно создать три карточки и увидеть их порядок до начала исполнения.

Открываем визуальную Kanban-доску

В PowerShell выполняю:

```
hermes dashboard
```

Откроется браузер с Hermes Dashboard.

Обычно адрес будет:

```
http://127.0.0.1:9119
```

В левой части интерфейса нахожу раздел:

Kanban

и нажимаю его.

Hermes Dashboard имеет отдельную Kanban-вкладку с колонками состояний, фильтрами, приоритетами, assignee и обновлением состояния карточек в реальном времени.

В верхней части проверяю выбранную доску.

Если отображается другая, открываю переключатель Board и выбираю:

Process Lab

Проверяем режим Orchestration

В верхней части Kanban нахожу переключатель:

Orchestration

Для этой практики автоматическая декомпозиция нам не нужна.

Переключаю его в:

Manual

Здесь я хочу сам создать три конкретных задания и управлять их порядком, а не поручать Hermes автоматически разбивать их на дополнительные карточки.

Создаём первую задачу

Нахожу колонку:

Ready

В её заголовке нажимаю:

+

Открывается окно создания задачи.

В поле Title ввожу:

Проанализировать правила AGENTS.md

В поле Assignee выбираю:

process-worker

В поле Priority ставлю:

1

В Workspace выбираю работу с существующим каталогом проекта и указываю:

корень выбранного проекта (`\$projectRoot`)

Если интерфейс предлагает тип workspace, выбираю существующую директорию проекта, то есть `dir`.

В описание задачи вставляю:

Работай только в режиме чтения.

Прочитай корневой AGENTS.md текущего проекта.

Кратко перечисли применимые правила проекта и безопасности.

Никакие файлы не изменяй.

После проверки заверши Kanban-задачу через `kanban_complete` и в `summary` укажи найденные ограничения.

Нажимаю кнопку создания задачи.

Карточка должна появиться в колонке `Ready`.

Создаём вторую задачу

Снова нажимаю:

+

в заголовке `Ready`.

Title:

Найти использования LegacyPaymentGateway

Assignee:

process-worker

Priority:

5

Workspace:

корень выбранного проекта (`\$projectRoot`)

Описание:

Работай только в режиме чтения.

В текущем проекте найди использования LegacyPaymentGateway внутри demo_codebase.

Укажи найденные файлы и места использования.

Ничего не изменяй.

После проверки заверши Kanban-задачу через kanban_complete и передай результат в summary.

Создаю карточку.

Создаём третью задачу

Ещё раз нажимаю  в колонке **Ready**.

Title:

Проверить demo_codebase линтером

Assignee:

process-worker

Priority:

10

Workspace:

корень выбранного проекта (``$projectRoot``)

Описание:

Работай только в режиме чтения.

Проверь Python-код в `demo_codebase` с помощью `Ruff`.

Используй существующее виртуальное окружение проекта.

Ничего автоматически не исправляй и не используй `--fix`.

Сообщи, прошла ли проверка, и укажи найденные проблемы.

После проверки заверши Kanban-задачу через `kanban_complete`.

Создаю карточку.

Смотрим настоящую очередь

Теперь у меня в `Ready` находятся три карточки.

Я ещё не запустил dispatcher, поэтому ни одна из них не должна находиться в `Running`.

Обращаю внимание на порядок.

Карточка с приоритетом:

10

должна находиться выше карточки с приоритетом `5`, а карточка с приоритетом `1` — ниже.

В Hermes очередь `ready` сортируется прежде всего по убыванию `priority`. При одинаковом приоритете раньше выбирается более старая задача. Этот же порядок используется dispatcher при выборе следующей работы.

Изменяем приоритет уже созданной задачи

Теперь покажу, что оператор может вмешиваться в очередь.

Нажимаю карточку:

```
Проанализировать правила AGENTS.md
```

Справа открывается drawer карточки.

Нахожу строку с Assignee и Priority. Нажимаю на значение Priority.

Вместо:

```
1
```

ставлю:

```
20
```

Сохраняю изменение и закрываю карточку.

Теперь смотрю обратно на колонку `Ready`.

Карточка должна переместиться наверх, потому что её приоритет теперь выше остальных.

Получился новый порядок:

```
20
10
5
```

Изменение приоритета выполняю через Dashboard. Набор параметров CLI может отличаться между версиями Hermes, поэтому при наличии отдельной команды изменения приоритета её также можно использовать.

Проверяем очередь из терминала

Открываю ещё одно окно PowerShell.

Выполняю:

```
hermes kanban boards switch process-lab
```

Теперь:

```
hermes kanban list --status ready --sort priority
```

Я должен увидеть наши три задания в порядке приоритетов.

Дополнительно выполняю:

```
hermes kanban stats
```

Здесь Hermes показывает количество задач по состояниям и assignee.

Пока ожидаю три `ready` и ноль `running`.

Запускаем dispatcher

Теперь очередь готова.

Открываю новое окно PowerShell и выполняю:

```
hermes gateway run
```

Это окно оставляю открытым.

Gateway содержит встроенный Kanban dispatcher. По умолчанию dispatcher периодически просматривает очередь и запускает подходящие задачи. Стандартный интервал составляет 60 секунд, но через Dashboard его можно запустить немедленно.

Возвращаюсь в браузер с Kanban.

В верхней панели нажимаю:

```
Nudge dispatcher
```

Эта кнопка заставляет dispatcher выполнить проход по очереди прямо сейчас, без ожидания следующего периодического tick.

Наблюдаем первый запуск

После обновления состояния одна карточка должна перейти из:

```
Ready
```

в:

```
In Progress
```

или `Running`, в зависимости от отображаемого интерфейсом названия колонки.

Поскольку `kanban.max_in_progress` равен `1`, одновременно должна работать только одна карточка.

И поскольку мы изменили Priority первой карточки на `20`, именно:

```
Проанализировать правила AGENTS.md
```

должна запускаться первой.

Карточки с приоритетами `10` и `5` остаются в очереди.

Таким образом, я одновременно проверил два механизма: приоритет определяет порядок, а квота ограничивает параллельность.

Открываем работающую карточку

Нажимаю карточку в `In Progress`.

Справа появляется её подробное представление.

Здесь я смотрю текущий assignee, статус, события и информацию о запуске.

Worker, который dispatcher создаёт для карточки, получает специальный Kanban-контекст и сам работает через `kanban_show`, `kanban_heartbeat`, `kanban_complete` и другие lifecycle-tools. Эти инструменты автоматически добавляются worker-процессу; вручную заставлять его выполнять `hermes kanban` через терминал не требуется.

Открываем живой мониторинг в терминале

Теперь в ещё одном PowerShell выполняю:

```
hermes kanban boards switch process-lab
```

Затем:

```
hermes kanban watch --assignee process-worker --interval 1
```

Это окно не закрываю.

Теперь здесь отображаются события очереди по мере их появления.

Когда задача будет взята, завершена или заблокирована, соответствующее событие появится в терминале. `hermes kanban watch` предназначен именно для live-наблюдения за событиями доски.

Остановить просмотр можно:

```
Ctrl + C
```

Смотрим лог конкретного worker

На карточке или через:

```
hermes kanban list
```

нахожу ID текущей задачи.

Например, он может выглядеть как:

```
t_ab12cd
```

Тогда выполняю:

```
hermes kanban log t_ab12cd
```

Так я вижу журнал работы worker.

Если мне интересна история запусков карточки, использую:

```
hermes kanban runs t_ab12cd
```

А подробное состояние самой задачи:

```
hermes kanban show t_ab12cd
```

Hermes хранит отдельный run для каждой попытки выполнения задачи, поэтому повторный запуск не уничтожает информацию о предыдущей попытке.

Наблюдаем завершение первой задачи

Когда агент закончит анализ `AGENTS.md`, он должен вызвать:

```
kanban_complete
```

Карточка автоматически переместится в:

```
Done
```

Я нажимаю её.

Внизу drawer нахожу Run History.

Здесь должна отображаться завершённая попытка, профиль:

```
process-worker
```

продолжительность и summary агента.

Hermes сохраняет summary и metadata выполнения именно в истории run, поэтому результат работы остаётся доступен после завершения worker.

Запускаем следующую задачу

После освобождения единственного разрешённого рабочего слота dispatcher сможет взять следующую карточку.

Можно подождать очередной tick.

Для занятия быстрее снова нажимаю:

```
Nudge dispatcher
```

Следующей должна запуститься карточка с Priority:

```
10
```

то есть:

```
Проверить demo_codebase линтером
```

Карточка с Priority 5 продолжает ждать.

Снова открываю её и наблюдаю состояние.

Временно блокируем последнюю задачу

Теперь я хочу показать, что оператор может убрать работу из очереди, не удаляя её.

Пока задача:

```
Найти использования LegacyPaymentGateway
```

остаётся в **Ready**, нажимаю на её карточку и копирую ID.

В PowerShell выполняю:

```
hermes kanban block <ID_ЗАДАЧИ> "Временно приостановлено преподавателем для демонстрации управ-
```

```
ления очередь"
```

Карточка перемещается в:

```
Blocked
```

Теперь dispatcher не должен её запускать.

Я могу нажать `Nudge dispatcher`, но заблокированная задача всё равно не перейдёт в `Running`.

Это важная часть управления процессами: задача не удалена и её история не потеряна, но она временно исключена из выполнения.

Возвращаем заблокированную задачу

Теперь разрешаю работу продолжить:

```
hermes kanban unblock <ID_ЗАДАЧИ>
```

Задача снова становится доступной для выполнения.

В зависимости от её условий она вернётся в рабочую очередь.

Нажимаю в Dashboard:

```
Nudge dispatcher
```

Если предыдущая задача уже завершилась и квота свободна, `LegacyPaymentGateway` переходит в `Running`.

Смотрим общую статистику

В терминале выполняю:

```
hermes kanban stats
```

Здесь я вижу распределение задач по состояниям.

В процессе занятия значения могут выглядеть примерно так: часть задач уже `done`, одна `running`, остальные `ready` или `blocked`.

Но я ориентируюсь не на конкретные цифры, а на фактическое состояние нашей очереди.

Для просмотра только работающих задач выполняю:

```
hermes kanban list --status running
```

Для ожидающих:

```
hermes kanban list --status ready --sort priority
```

Для завершённых:

```
hermes kanban list --status done
```

Используем фильтры Dashboard

Возвращаюсь в браузер.

В верхней панели Kanban нахожу поле Assignee.

Выбираю:

```
process-worker
```

Теперь отображаются только задания этого рабочего профиля.

Если включён параметр:

```
Lanes by profile
```

в колонке текущей работы Hermes группирует running-задачи по профилям. Для нашей практики профиль только один, но при нескольких агентах это позволяет сразу увидеть нагрузку каждого worker.

Проверяем окончательное состояние

После выполнения всех трёх карточек выполняю:

```
hermes kanban list
```

и:

```
hermes kanban stats
```

Все три учебные задачи должны оказаться в **done**.

Затем в Dashboard открываю каждую карточку и смотрю Run History.

Первая должна содержать результат анализа `AGENTS.md`.

Вторая должна содержать результат Ruff.

Третья должна содержать места использования `LegacyPaymentGateway`.

При этом никакие задачи не выполнялись как часть текущего диалога Hermes Desktop. Они существовали как самостоятельные элементы долговечной очереди, а dispatcher создавал для них worker-процессы.

Что мы фактически проверили

В этой практике я использовал Hermes уже как систему управления работами, а не как один чат с моделью.

Kanban board стала нашей очередью задач. `Ready` означал, что задание готово к исполнению. `Running` показывал реальную работу worker. `Blocked` позволил временно остановить карточку без её удаления. `Done` зафиксировал результат.

Приоритет управлял порядком выполнения. В очереди Hermes карточки с большим числом Priority выбираются раньше карточек с меньшим приоритетом. При одинаковом приоритете порядок определяется временем создания.

Параметр:

```
kanban.max_in_progress = 1
```

стал нашей квотой. Независимо от количества готовых задач Hermes не запускал больше одной одновременно.

Для мониторинга мы использовали одновременно визуальную Kanban-доску и команды:

```
hermes kanban watch
hermes kanban list
hermes kanban stats
hermes kanban show <ID>
hermes kanban runs <ID>
hermes kanban log <ID>
```

Таким образом, состояние работы определялось не сообщением модели «я закончил», а инфраструктурой Hermes. Именно это и превращает набор отдельных агентов в управляемый процесс.
